

02-dialog

Dialog

There are three types of dialog in playwright,

- alert
- confirm
- prompt

By default playwright **auto dismisses** every dialog. To actually handle them we need to listen for the `dialog` event using an `dialog handler`.

```
page.on("dialog", async (d) => {
  await d.accept();
  await d.dismiss();
  d.type();
  d.message();
  d.defaultValue();
});
```

*Important: the `dialog handler` is always written **before** the action that triggers the alert, so it does not miss the dialog.*

*Important: inside the handler you **must** call `d.accept()` or `d.dismiss()`. If you add a handler but never accept/dismiss, the dialog stays open and the test hangs.*

Events used to listen dialog

- `page.on()` ⇒ listens for the event continuously once activated
- `page.once()` ⇒ listens only once for the defined event
- `page.waitForEvent("dialog")` ⇒ waits for and returns the dialog (used when you want to listen inline)

Different methods in dialog

- `d.type()` ⇒ gives type of dialog (`alert` / `confirm` / `prompt`)
- `d.message()` ⇒ gives text present on the dialog
- `d.defaultValue()` ⇒ gives the default value of a `prompt` dialog
- `d.accept()` ⇒ clicks OK on the dialog
- `d.dismiss()` ⇒ dismisses (clicks Cancel) on the dialog

- `d.accept("john")` ⇒ sends text into a `prompt` input then accepts

simple alert

Simple alert with only an `OK` button. Used to show a warning message.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)⇒ {
  expect(d.type()).toContain("alert");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  // await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

confirm alert

Accept or reject type alert with `OK` and `Cancel` buttons.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)⇒ {
  expect(d.type()).toContain("confirm");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  // await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

prompt alert

Accept or reject type alert where the user can also send a `custom message` into an input.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)⇒ {
  expect(d.type()).toContain("prompt");
  expect(d.message()).toContain("alert message text");

  await d.accept("Kundalik"); // text passed into the prompt input
});

// click on alert button
await page.locator("#alert-btn").click();
```

Dialog Handler

Using the dialog handler we can interact with alerts. We can inspect the dialog `type` and `message`, then either `accept` or `dismiss` them, and for a `prompt` alert we can provide an input value.

Interview Questions

Q - Why do we add the dialog handler before clicking the alert button?

Ans:-

- The alert appears immediately when we click the alert button.
- JS is async in nature, so the alert can fire before a handler placed after the click is ready.
- To handle the alert we must already be listening on the `dialog` event; if the handler is placed after the click, we may lose the dialog.
- So the dialog handler is always placed before clicking the alert.

Q - What is the difference between `page.on()` and `page.once()`?

Ans:-

- `page.on()` ⇒ once activated, listens for **every** dialog
- `page.once()` ⇒ listens **only once** after it is activated

Q - What happens if we don't handle a dialog?

Ans:-

- Playwright auto dismisses it by default.
- But if a handler is registered and we never call `accept()` / `dismiss()`, the test will hang/timeout.

Alerts in Selenium

In Selenium Java an alert is handled using the `Alert` interface. We first switch to the alert, then act on it.

```
// switch to the alert
Alert alert = driver.switchTo().alert();

alert.accept();           // click OK
alert.dismiss();          // click Cancel
alert.getText();           // read the alert message
alert.sendKeys("kundalik"); // type into a prompt alert
```

Playwright vs Selenium — Alert Handling

Aspect	Playwright	Selenium (Java)
Model	Event driven (listen for <code>dialog</code>)	Switch to alert (<code>switchTo().alert()</code>)
When to set up	Handler before the trigger click	Switch after the alert appears
Default behaviour	Auto dismisses unless handled	Stays open until handled (else error)
Get type	<code>d.type()</code>	No direct method
Read message	<code>d.message()</code>	<code>alert.getText()</code>
Accept / OK	<code>d.accept()</code>	<code>alert.accept()</code>
Dismiss / Cancel	<code>d.dismiss()</code>	<code>alert.dismiss()</code>
Prompt input	<code>d.accept("text")</code>	<code>alert.sendKeys("text")</code> then <code>accept()</code>
Not handled	Auto dismissed	<code>UnhandledAlertException</code>

Playwright → Selenium (Java) Method Mapping

Playwright	Selenium (Java) equivalent
<code>page.on("dialog", handler)</code>	<code>driver.switchTo().alert()</code>
<code>d.message()</code>	<code>alert.getText()</code>
<code>d.accept()</code>	<code>alert.accept()</code>
<code>d.dismiss()</code>	<code>alert.dismiss()</code>
<code>d.accept("text")</code>	<code>alert.sendKeys("text")</code> then <code>alert.accept()</code>
<code>d.type()</code>	no direct equivalent
<code>d.defaultValue()</code>	no direct equivalent